

@Keyur's Code_Hub

Python

Unit – 2

○ Functions, Exception, Modules and Files

1. Difference Between a Function and a Method

- **Function:** A block of reusable code that performs a specific task. It can be defined using the `def` keyword.
- **Method:** A function associated with an object. Defined inside a class and acts on an instance of that class.

Example:

```
# Function
def greet():
    return "Hello"

print(greet()) # Function call

# Method inside a class
class Person:
    def say_hello(self):
        return "Hello from method"

p = Person()
print(p.say_hello()) # Method call
```

2. Defining a Function

A function is defined using `def` keyword.

```
def add(a, b):

    return a + b
```

3. Calling a Function

A function is executed when it is called.

```
result = add(5, 3)

print(result) # Output: 8
```

4. Returning Results from a Function

Functions use the return statement to send values back.

```
def square(n):

    return n * n

print(square(4)) # Output: 16
```

5. Returning Multiple Values from a Function

Python allows returning multiple values as a tuple.

```
def stats(a, b):

    return a + b, a - b, a * b

sum_, diff, prod = stats(10, 5)

print(sum_, diff, prod) # Output: 15 5 50
```

6. Functions are First-Class Objects

Functions can be assigned to variables, passed as arguments, and returned from other functions.

```
def greet(name):  
    return f"Hello, {name}"  
  
greeting = greet # Assigning function to a variable  
print(greeting("Keyur")) # Output: Hello, Keyur
```

7. Pass by Object Reference

Python uses **pass-by-object-reference**, meaning immutable objects cannot be changed, but mutable objects can.

```
def modify_list(lst):  
    lst.append(4)  
  
my_list = [1, 2, 3]  
modify_list(my_list)  
print(my_list) # Output: [1, 2, 3, 4]
```

8. Formal and Actual Arguments

- **Formal arguments:** Parameters in the function definition.
- **Actual arguments:** Values passed during a function call.

```
def greet(name): # Formal argument  
    return f"Hello, {name}"  
  
print(greet("Alice")) # Actual argument
```

9. Positional Arguments

Values are assigned based on their position.

```
def person(name, age):  
    print(name, age)  
  
person("Alice", 25) # Output: Alice 25
```

10. Keyword Arguments

Explicitly mention parameter names to avoid confusion.

```
person(age=25, name="Alice") # Output: Alice 25
```

11. Default Arguments

Provide a default value for parameters.

```
def greet(name="Guest"):  
    return f"Hello, {name}"  
  
print(greet())    # Output: Hello, Guest  
print(greet("Keyur")) # Output: Hello, Keyur
```

12. Variable-Length Arguments

- ***args**: Accepts multiple positional arguments.
- ****kwargs**: Accepts multiple keyword arguments.

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1, 2, 3, 4)) # Output: 10  
  
def details(**info):  
    print(info)  
  
details(name="Alice", age=25) # Output: {'name': 'Alice', 'age': 25}
```

13. Local and Global Variables

- **Local variable:** Defined inside a function.
- **Global variable:** Accessible throughout the program.

```
x = 10 # Global variable  
  
def my_function():  
    y = 5 # Local variable  
    print(x + y)  
  
my_function() # Output: 15
```

14. The Global Keyword

Used to modify global variables inside functions.

```
count = 0

def increment():
    global count
    count += 1

increment()
print(count) # Output: 1
```

15. Passing a Group of Elements to a Function

Lists, tuples, or dictionaries can be passed to functions.

```
def print_items(items):

    for item in items:

        print(item)

fruits = ["Apple", "Banana", "Cherry"]

print_items(fruits)
```

16. Recursive Functions

A function that calls itself.

```
def factorial(n):

    if n == 1:

        return 1

    return n * factorial(n - 1)

print(factorial(5)) # Output: 120
```

17. Anonymous Functions (Lambdas)

Lambdas are one-line functions.

```
square = lambda x: x * x  
  
print(square(5)) # Output: 25
```

Using Lambdas with filter()

```
numbers = [1, 2, 3, 4, 5]  
  
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))  
  
print(even_numbers) # Output: [2, 4]
```

Using Lambdas with map()

```
squared = list(map(lambda x: x * x, numbers))  
  
print(squared) # Output: [1, 4, 9, 16, 25]
```

Using Lambdas with reduce()

```
from functools import reduce  
  
product = reduce(lambda x, y: x * y, numbers)  
  
print(product) # Output: 120
```


18. Function Decorators

Decorators modify the behavior of functions.

```
def decorator(func):  
  
    def wrapper():  
  
        print("Before function call")  
  
        func()  
  
        print("After function call")  
  
    return wrapper  
  
@decorator  
  
def say_hello():  
  
    print("Hello!")  
  
say_hello()
```

19. Generators

Generators use yield and are memory efficient.

```
def my_generator():  
  
    for i in range(3):  
  
        yield i  
  
gen = my_generator()  
  
print(next(gen)) # Output: 0  
  
print(next(gen)) # Output: 1
```

20. Structured Programming

Python follows structured programming with modular code and functions.

```
def add(a, b):  
    return a + b  
  
print(add(2, 3)) # Output: 5
```

21. Creating Our Own Modules

Create a module named mymodule.py:

```
def greet(name):  
    return f"Hello, {name}"  
  
Import and use it:  
  
import mymodule  
  
print(mymodule.greet("Alice"))
```

22. The Special Variable __name__

The __name__ variable determines if the script is being run directly or imported as a module.

```
if __name__ == "__main__":  
    print("Executed as main script")
```

When run directly: Executed as main script When imported: Nothing prints.

Exceptions in Python: A Detailed Guide with Examples

1. Errors in a Python Program

Errors can be classified into three types:

1. **Compile-Time Errors (Syntax Errors)**
2. **Runtime Errors (Exceptions)**
3. **Logical Errors**

1.1 Compile-Time Errors (Syntax Errors)

These occur when the Python interpreter encounters incorrect syntax.

```
print("Hello" # Missing closing parenthesis - SyntaxError
```

Output:

```
SyntaxError: unexpected EOF while parsing
```

Python doesn't allow the program to run until syntax errors are fixed.

1.2 Runtime Errors (Exceptions)

These occur during execution due to invalid operations (e.g., division by zero, accessing an undefined variable).

```
x = 10 / 0 # ZeroDivisionError
```

Output:

```
ZeroDivisionError: division by zero
```

1.3 Logical Errors

Logical errors don't cause crashes but lead to incorrect results.

```
def add(a, b):  
    return a - b # Wrong operation (logical mistake)  
  
print(add(5, 3)) # Output: 2 (Expected: 8)
```

2. Exceptions in Python

Exceptions are runtime errors that can be handled using try-except.

Example:

```
try:  
    num = int(input("Enter a number: "))  
    result = 10 / num # Possible ZeroDivisionError  
    print(result)  
except ZeroDivisionError:  
    print("Cannot divide by zero!")
```

Input & Output:

Enter a number: 0

Cannot divide by zero!

3. Exception Handling

Python provides the try-except block to handle exceptions and prevent program crashes.

3.1 The try-except Block

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("Error: Division by zero is not allowed!")
```

Output:

Error: Division by zero is not allowed!

3.2 Handling Multiple Exceptions

```
try:  
    x = int("abc") # ValueError  
except (ValueError, TypeError):  
    print("Invalid input!")
```

Output:

Invalid input!

3.3 Catching Any Exception (Exception class)

```
try:  
    x = 10 / 0  
except Exception as e:  
    print(f"An error occurred: {e}")
```

Output:

An error occurred: division by zero

3.4 The else Clause

Executes if no exception occurs.

```
try:
    num = 5 / 1
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("No error occurred!") # Runs when no error happens
```

Output:

No error occurred!

3.5 The finally Clause

Executes no matter what.

```
try:
    num = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
finally:
    print("Execution finished.") # Runs no matter what
```

Output:

Cannot divide by zero!

Execution finished.

4. Types of Exceptions

Python has many built-in exceptions:

- `ZeroDivisionError`
- `ValueError`
- `TypeError`
- `IndexError`
- `KeyError`
- `FileNotFoundError`
- `ImportError`
- `AttributeError`
- `NameError`
- `IOError`

Example: `IndexError`

```
my_list = [1, 2, 3]

print(my_list[5]) # IndexError: list index out of range
```

Example: `KeyError`

```
data = {"name": "Alice"}

print(data["age"]) # KeyError: 'age' (Key does not exist)
```

5. The assert Statement

The `assert` statement is used for debugging; it raises an `AssertionError` if a condition is `False`.

```
x = -1

assert x > 0, "x must be positive!"
```

Output:

```
AssertionError: x must be positive!
```

6. User-Defined Exceptions

We can create custom exceptions by inheriting from `Exception`.

```
class NegativeNumberError(Exception):  
    pass  
  
def check_positive(n):  
    if n < 0:  
        raise NegativeNumberError("Negative numbers are not allowed")  
  
try:  
    check_positive(-5)  
except NegativeNumberError as e:  
    print(e)  
Output:  
Negative numbers are not allowed
```

7. Logging Exceptions

Logging helps store exception details for debugging.

```
import logging  
  
logging.basicConfig(filename="errors.log", level=logging.ERROR)  
  
try:  
    x = 10 / 0  
except ZeroDivisionError as e:  
    logging.error(f"Exception occurred: {e}")  
  
This logs the error in errors.log.
```


Concept	Explanation	Example
Compile-Time Errors	Syntax issues	<code>print("Hello"</code>
Runtime Errors	Errors during execution	<code>10 / 0</code>
Logical Errors	Wrong logic without errors	<code>return a - b</code> instead of <code>return a + b</code>
Exception Handling	Using try-except to catch errors	<code>try: x = 10 / 0 except:</code>
Types of Exceptions	Various built-in exceptions	<code>IndexError</code> , <code>KeyError</code> , <code>ValueError</code>
The assert Statement	Used for debugging	<code>assert x > 0, "x must be positive!"</code>
User-Defined Exceptions	Creating custom errors	<code>class MyError(Exception): pass</code>
Logging	Saves errors to a file	<code>logging.error("Error message")</code>